

Documentazione: Sistema di Gestione Ospedaliera

Componenti del Gruppo:

Iazzetta Angelo Karol,
Di Domenico Lorenzo,
Mottola Giuseppe.

Anno Accademico: 2025/2026

Indice

1. Capitolo 1: Descrizione e Analisi del Progetto
2. Capitolo 2: Progettazione Concettuale
3. Capitolo 3: Progettazione della Soluzione
4. Capitolo 4: Architettura dei Package
5. Capitolo 5: Features e Controlli
6. Capitolo 6: Manuale d'uso dell'applicazione

Capitolo 1

Descrizione e Analisi del Progetto

1.1 Introduzione

Il presente documento descrive l'architettura software del progetto "Sistema Gestione Ospedaliera". L'obiettivo del sistema informativo è fornire un applicativo Java dotato di interfaccia grafica (GUI) in Swing e supportato da una base di dati relazionale, in grado di gestire in modo efficiente le risorse strutturali, i ricoveri dei pazienti e i turni del personale medico.

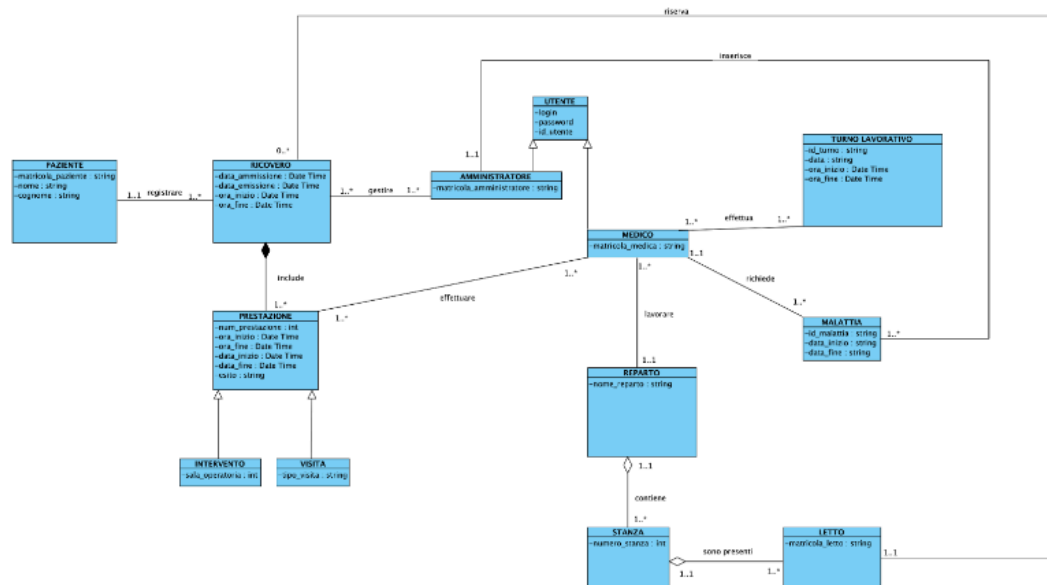
L'applicazione è strutturata secondo il pattern architetturale MVC (Model-View-Controller) per garantire una separazione tra la logica di dominio, il flusso applicativo e l'interfaccia utente. Il sistema pone particolare attenzione sulla

coerenza dei dati, impedendo sovrapposizioni temporali nell'allocazione dei letti e nella programmazione delle prestazioni mediche. Inoltre, essendo il progetto sviluppato da un gruppo di tre persone, il sistema implementa la gestione avanzata dello stato di "Malattia" del personale, automatizzando la ricerca di medici sostitutivi per i turni scoperti.

Capitolo 2

Progettazione Concettuale

2.1 Class Diagram del dominio del problema

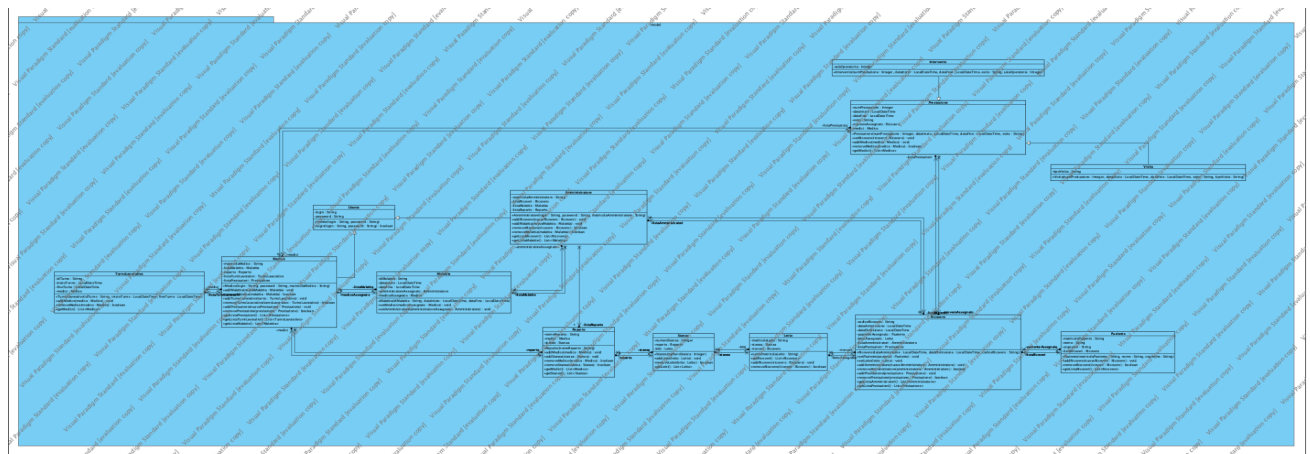


Capitolo 3

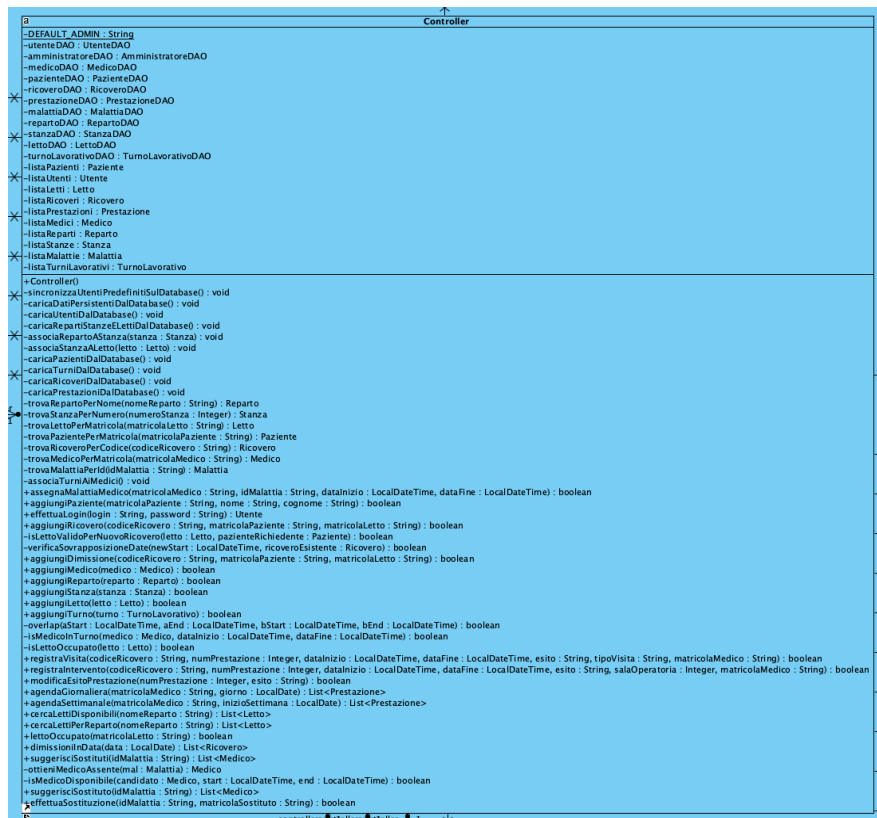
Progettazione della Soluzione

Class Diagram del dominio della soluzione

3.1 PACKAGE MODEL



3.2 PACKAGE CONTROLLER



PACKAGE GUI

```

Amministrazione
DATE_FORMATTER : DateFormatter
-controller : Controller
-panel : JPanel
-etichettaLabel : JLabel
-pazienteMatricolaField : JTextField
-pazienteNomeField : JTextField
-pazienteCognomeField : JTextField
-aggiungiPazienteButton : JButton
-ricoveroCodiceField : JTextField
-ricoveroPazienteField : JTextField
-ricoveroLettoField : JTextField
-registraRicoveroButton : JButton
-registraDimissioniButton : JButton
-dataDimissioniField : JTextField
-ricercaDimissioniButton : JButton
-dimissioniList : JList<String>
-aggiungiCensitiButton : JButton
-aggiungiSostituzioni : JButton
-aggiungiSostituzioniButton : JButton
-loggaButton : JButton
-dimissioniListModel : DefaultListModel<String>
+Amministrazione()
+Amministrazione(controller : Controller)
+getContentPane() : JComponent
+configuraUI() : void
+aggiungiPaziente() : void
+aggiungiRicovero() : void
+registraDimissioni() : void
+ricercaDimissioni() : void
+aggiungiCensiti() : void
+descrittoriRicovero(ricovero : Ricovero) : String
+descrittoriMedico(medico : Medico) : String
+leggiField : JTextField : String
+aggiungiMedico(messaggio : String) : void
+mostraAvviso(messaggio : String) : void
+main : String : void
+setUpUIS() : void
+setUpFont($$ (fontName : String, style : int, size : int, currentFont : Font) : Font
+getLabelComponent($$) : JComponent

```

```

GestioneLetto
-controller : Controller
-noaPanel : JPanel
-registraField : JTextField
-lettoListModel : DefaultListModel<Letto>
-lettoList : JList<Letto>
-countLabel : JLabel
-etichettaLabel : JLabel
+GestioneLetto()
+GestioneLetto(controller : Controller)
+getContentPane() : JComponent
+main : String : void
+configuraUI() : void
+refreshList() : void

```

```

login
-controller : Controller
-panel : JPanel
-usernameField : JTextField
-passwordField : JPasswordField
-ACCEDIButton : JButton
+login()
+login(controller : Controller)
+getContentPane() : JComponent
+configuraUI() : void
+readUserFromDB() : String
+diagnosticaAccesso() : void
+aggiungiUser(utente : Utente) : void
+aggiungiUserFromDB() : JComponent, titolo : String, dimensioniMinime : Dimension) : void
+showLogin(controller : Controller) : void
+main : String : void
+setUpUIS() : void
+setUpFont($$ (fontName : String, style : int, size : int, currentFont : Font) : Font
+getLabelComponent($$) : JComponent

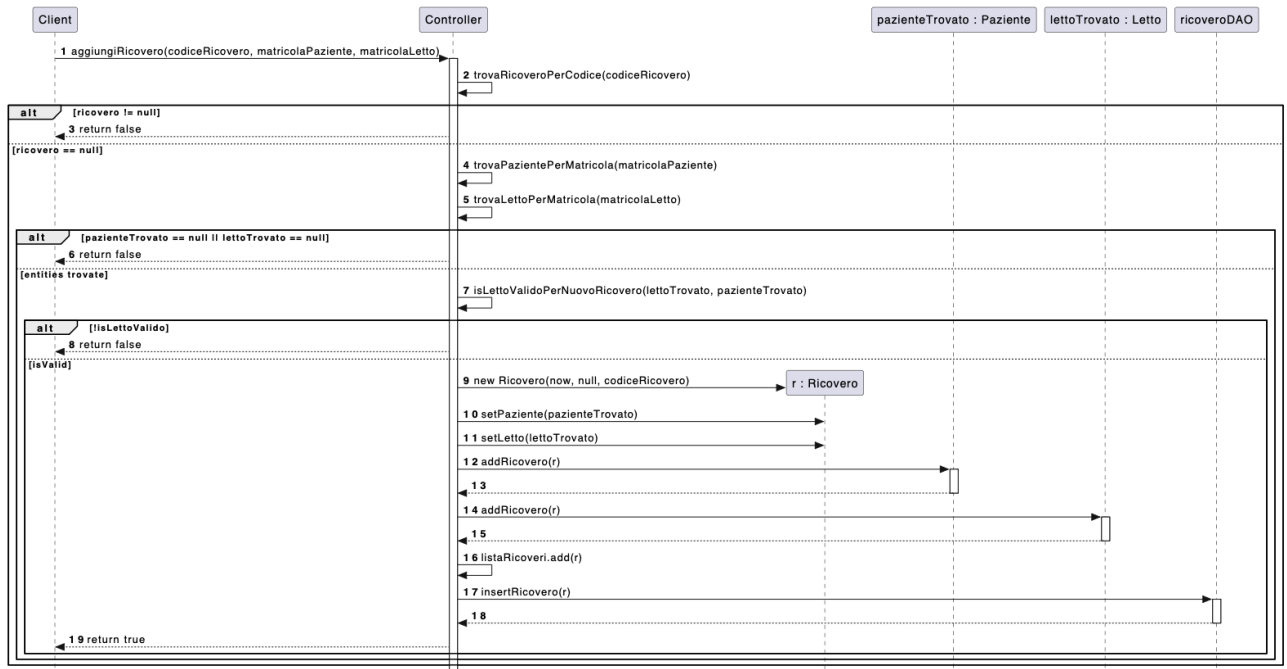
```

```

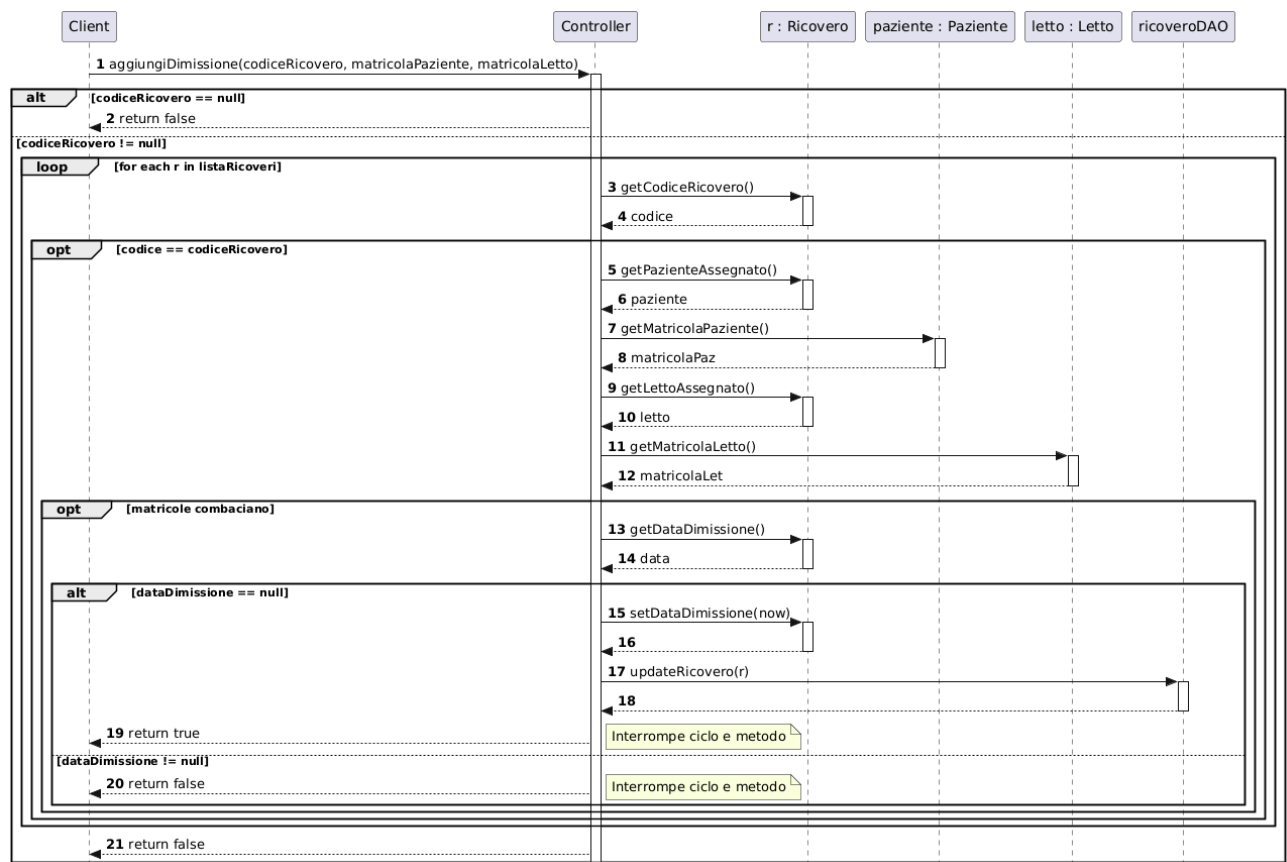
Medico
DATE_FORMATTER : DateFormatter
DATE_TIME_FORMATTER : DateFormatter
-controller : Controller
-panel : JPanel
-etichettaLabel : JLabel
-ricoveroMatricolaField : JTextField
-ricoveroCodiceField : JTextField
-numeroPrestazioniField : JTextField
-dataPrestazioneField : JTextField
-dataFineField : JTextField
-medicoField : JTextField
-tipologiaField : JTextField
-ricercaPrestazioniField : JTextField
-registraPrestazioneButton : JButton
-registraInterventoButton : JButton
-aggiungiPrestazioneButton : JButton
-aggiungiDataField : JTextField
-aggiungiCensitiButton : JButton
-aggiungiMensualiButton : JButton
-aggiungiList : JList<String>
-loggaButton : JButton
-aggiungiListModel : DefaultListModel<String>
+Medico()
+Medico(controller : Controller)
+getContentPane() : JComponent
+configuraUI() : void
+elaboraPrestazioniIntervento : boolean) : void
+aggiungiPrestazione() : void
+aggiungiIntervento() : void
+aggiungiCensiti() : void
+aggiungiMensuali() : void
+aggiungiPrestazioni() : List<Prestazione>, emptyMessage : String) : void
+logga() : void
+descrittoriPrestazioni(prestazione : Prestazione) : String
+leggiField : JTextField : String
+aggiungiMedico(messaggio : String) : void
+mostraAvviso(messaggio : String) : void
+main : String : void
+setUpUIS() : void
+setUpFont($$ (fontName : String, style : int, size : int, currentFont : Font) : Font
+getLabelComponent($$) : JComponent

```

3.3 Sequence Diagram del metodo aggiungiRicovero.



3.4 Sequence Diagram del metodo aggiungiDimissione.



CAPITOLO 4

Architettura dei Package

Il codice sorgente è stato suddiviso in tre package per rispettare i principi di modularità e coesione:

- **model:** Contiene le entità del dominio e la logica di business fondamentale.
- **controller:** Gestisce il flusso dell'applicazione, intercetta le azioni dell'utente e gestisce la comunicazione tra l'interfaccia e il modello.
- **gui:** Contiene le classi dedicate all'Interfaccia Grafica Utente (Graphical User Interface).

4.1 Analisi dei Componenti Logici

Il package model rappresenta il cuore logico dell'applicazione, in cui sono state implementate le classi che mappano le entità reali dell'ospedale, avvalendosi dei concetti di ereditarietà e polimorfismo.

4.2 Gestione delle Utenze e dei Ruoli

- **Utente:** Classe base che incapsula i concetti fondamentali di autenticazione (login e password) necessari per l'accesso univoco al sistema.
- **Amministratore:** Sottoclasse di Utente che gestisce il personale amministrativo. Ha i privilegi per gestire le anagrafiche, allocare i ricoveri nei letti, gestire le assenze per malattia e visualizzare le statistiche dei pazienti in dimissione.
- **Medico:** Sottoclasse di Utente che rappresenta il personale sanitario. È associato in modo esclusivo a un singolo Reparto. Possiede liste di TurnoLavorativo e Prestazione, gestite in modo da evitare accavallamenti temporali.
- **Paziente:** Gestisce l'anagrafica del paziente ospedaliero, entità fondamentale per la registrazione dei ricoveri.

4.3 Gestione Strutturale

L'infrastruttura fisica dell'ospedale è modellata in modo gerarchico:

- **Reparto:** Raggruppa le stanze e il personale medico afferente.
- **Stanza:** Luogo dove sono contenuti i letti.
- **Letto:** Identifica la postazione univoca all'interno dell'intero ospedale.

4.4 Processi Clinici e Organizzativi

- **Ricovero:** Associa un paziente a un letto specifico per un intervallo di tempo (data/ora di inizio e dimissione).
- **TurnoLavorativo:** Definisce le fasce orarie programmate in cui un medico presta servizio nel proprio reparto.
- **Prestazione:** Classe astratta che rappresenta un'attività clinica erogata da un medico durante un ricovero. Viene specializzata in due sottoclassi concrete:
 - **Visita:** Contiene attributi specifici come il tipo Visita.
 - **Intervento:** Contiene dati strutturali aggiuntivi come il numero di Sala Operatoria.
- **Malattia:** Definisce il periodo di assenza di un medico e permette all'amministratore di eseguire la logica di ricerca per le sostituzioni.

Capitolo 5

Features e Controlli

Il package controller viene implementato attraverso il Controller, il quale agisce come gestore dello stato applicativo e come garante assoluto dell'integrità dei dati.

Le responsabilità della classe Controller includono:

- **Autenticazione e Inizializzazione:** Attraverso il metodo `effettuaLogin`, il controller verifica la validità delle credenziali ciclando la lista degli utenti registrati.
- **Gestione Ricoveri e Prevenzione Sovrapposizioni:** Il sistema implementa una logica di validazione temporale per l'assegnazione delle risorse. Il metodo `aggiungiRicovero` impedisce l'immatricolazione di un nuovo paziente su un Letto già occupato. La chiusura dell'assegnazione avviene tramite `aggiungiDimissione`.
- **Validazione delle Prestazioni Mediche:** I metodi `registraVisita` e `registraIntervento` assicurano che l'orario ricada in un `TurnoLavorativo` valido e che non ci siano sovrapposizioni nell'agenda del medico.
- **Motore di Ricerca Sostituzioni:** Il metodo `suggerisciSostituti` identifica i colleghi dello stesso reparto disponibili, escludendo chi ha già impegni durante l'intervallo di assenza.

- **Estrazione Dati e Viste:** Utilizza metodi come `agendaGiornaliera`, `agendaSettimanale` e `cercaLettiDisponibili` per popolare dinamicamente l'interfaccia.
 - Registrazione delle Dimissioni.
 - Gestione Malattie e ricerca sostituti.
 - **Gestione Letto:** Finestra di utilità per la ricerca rapida in tempo reale (tramite `DocumentListener`), che evidenzia in rosso i letti occupati e in verde quelli disponibili.
- **Medico (Dashboard Medico):** Ambiente dedicato al personale sanitario:
 - **Registrazione Prestazioni:** Pulsanti per registrare visite e interventi o aggiornarne l'esito.
 - **Agenda:** Visualizzazione cronologica degli impegni (giornaliera o settimanale) in una `JList`.

Capitolo 6

Manuale d'uso dell'applicazione:

L'applicazione offre un'interfaccia grafica (GUI) intuitiva e strutturata per facilitare la gestione e la consultazione dei dati

1) Schermata di Login

La schermata di accesso è la prima cosa che l'utente vede quando avvia l'applicazione. Serve a proteggere i dati dell'ospedale richiedendo l'inserimento di un Username e di una Password.



Quando l'utente clicca sul pulsante per accedere, il file `Controller.java` prende i dati inseriti e controlla nel database se sono corretti. Inoltre, il sistema verifica se l'utente

è un **Amministratore** o un **Medico**: a seconda del ruolo, il controller chiude la pagina di login e apre la dashboard corretta con le funzioni dedicate a quel profilo.

2) Dashboard Amministratore

La Dashboard è la pagina principale per l'utente Amministratore. È stata progettata come un unico pannello centrale da cui è possibile raggiungere tutte le funzioni di gestione della struttura ospedaliera.

The screenshot shows a web application window titled "Dashboard amministratore". The interface is divided into several sections:

- Header:** "Dashboard amministratore" with a subtitle "Gestisci pazienti, ricoveri, dimissioni, disponibilit  letti e sostituti."
- Gestione pazienti:** Contains input fields for "Matricola", "Nome", and "Cognome", followed by an "Aggiungi paziente" button.
- Ricoveri e dimissioni:** Contains input fields for "Codice ricovero", "Matricola paziente", and "Matricola letto", followed by "Registra ricovero" and "Registra dimissione" buttons.
- Dimissioni per data:** Contains a "Data" input field with the value "26/06/2026" and a "Cerca dimissioni" button, with a large empty box below for results.
- Strumenti:** A vertical list of buttons: "Ricerca letti disponibili", "Assegna malattia a medico", "Suggerisci sostituto", and "Logout".

Da questa schermata, l'amministratore pu  usare i vari pulsanti e menu per gestire i dati dei pazienti, registrare i ricoveri nei reparti, controllare i letti liberi o gestire le assenze dei medici per malattia. Praticamente fa da "registra" per smistare l'utente verso le varie sezioni del software.

2.1) Panoramica Gestione pazienti

Questa sezione serve a inserire un nuovo paziente all'interno del sistema dell'ospedale. L'interfaccia mette a disposizione dei campi di testo in cui digitare la Matricola (il codice identificativo), il Nome e il Cognome.

Quando si preme il tasto di conferma, il codice Java controlla prima di tutto che la matricola inserita non sia gi  stata assegnata a qualcun altro; se   tutto in regola, salva il nuovo paziente nel database in modo che sia pronto per essere ricoverato.

2.2) Panoramica Ricovero e dimissioni

Questa schermata permette di gestire l'assegnazione dei pazienti ai posti letto e la loro uscita dall'ospedale. Per inserire un ricovero, l'amministratore sceglie il paziente, seleziona un letto e imposta il periodo di tempo (il giorno e l'ora di inizio e la data delle dimissioni previste).

In questa fase il programma fa un controllo fondamentale: va a vedere nel database se quel letto è già occupato da qualcun altro in quelle stesse ore. Se il letto è già preso, l'app blocca l'operazione per evitare errori. Per aiutare visivamente l'utente, i letti occupati vengono mostrati con il codice colorato in **rosso**. Dalla stessa pagina si può anche registrare la dimissione effettiva quando un paziente lascia l'ospedale, liberando subito il letto.

2.3) Dimissioni per data

Questa funzione è un filtro utile per pianificare il lavoro della giornata. L'amministratore inserisce una data specifica (o seleziona il giorno corrente) e l'applicazione mostra in una tabella tutti i pazienti che devono essere dimessi o che sono stati dimessi in quel preciso giorno.

Il sistema ci riesce facendo una query al database che seleziona solo i ricoveri che hanno quella determinata data come giorno di dimissione.

2.4) Panoramica rapida Strumenti

Questa è una zona della dashboard che raccoglie delle funzioni rapide di supporto. Sono pensate per essere usate velocemente, permettendo all'amministratore di fare controlli al volo o piccole operazioni senza dover navigare in altre pagine complicate dell'applicazione.

2.4.1) Ricerca letti disponibili

Questa utility serve a trovare al volo un posto letto libero in caso di emergenza. L'amministratore seleziona il reparto che gli interessa da un menu a tendina (JComboBox) e l'applicazione mostra l'elenco dei letti vuoti. Il codice calcola questo elenco prendendo tutti i letti presenti in quel reparto e scartando quelli che risultano occupati da un ricovero ancora aperto.

2.4.2) Assegna malattia medico

Questa schermata serve quando un medico si ammala e non può andare al lavoro. L'amministratore seleziona il medico, inserisce il codice della malattia e imposta il giorno di inizio e di fine dell'assenza.

Il controller prende queste informazioni e le salva nel database, in modo che il sistema sappia che in quei giorni quel medico non sarà disponibile per i turni.

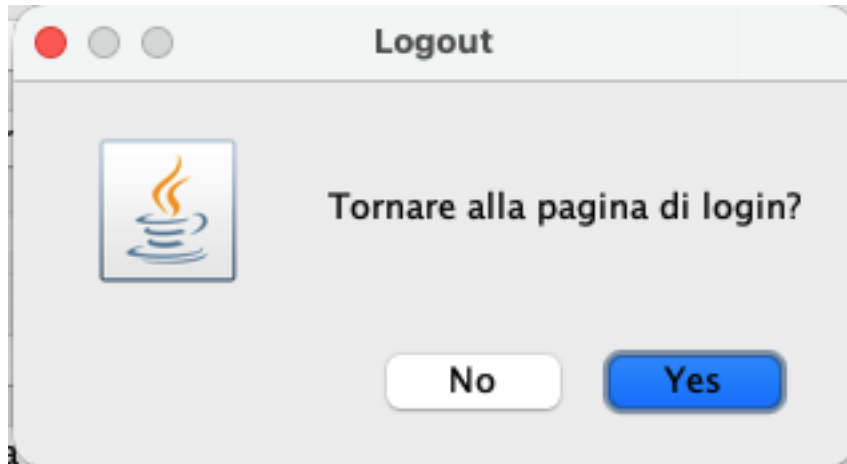
2.4.3) Suggestisci sostituto

Questa è una delle funzioni più avanzate del progetto: serve a trovare un medico che possa coprire i turni scoperti del collega malato. Selezionando l'assenza inserita prima, l'applicazione fa un controllo automatico e suggerisce una lista di medici idonei.

Per essere inserito nella lista dei sostituti, il software controlla due regole precise: il sostituto deve lavorare nello **stesso reparto** del medico assente e **non deve avere altri turni o visite già assegnate** in quelle stesse ore, evitando così che un medico si trovi in due posti contemporaneamente.

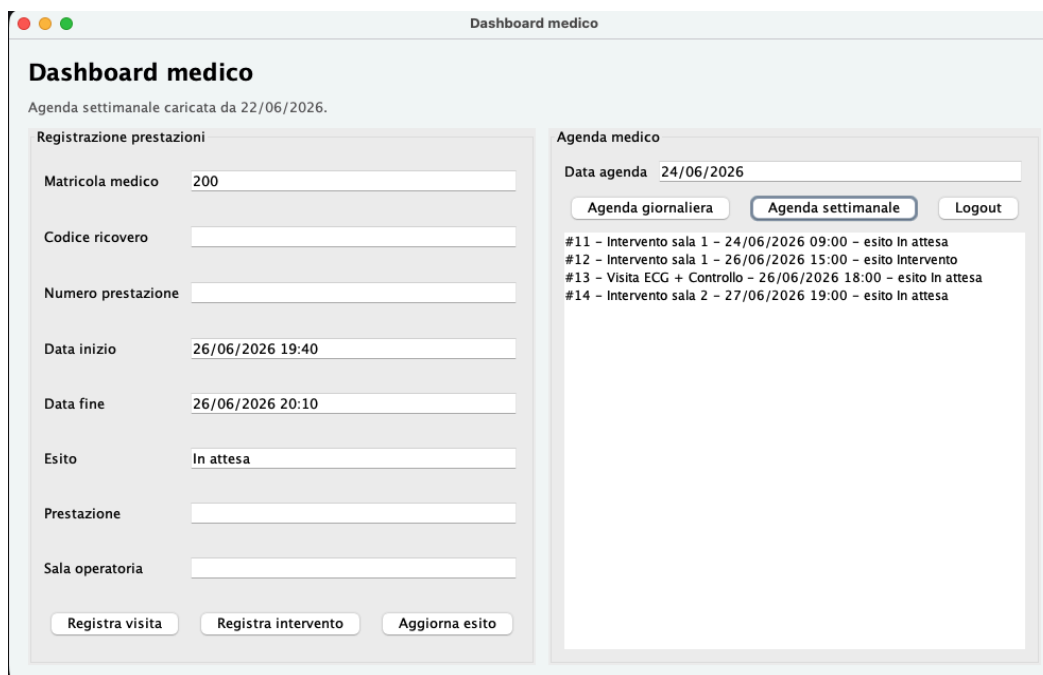
2.4.4) Logout

Questo pulsante serve a uscire in sicurezza dall'applicazione. Quando viene cliccato, il programma cancella i dati dell'utente dalla memoria, chiude la dashboard dell'amministratore e riapre la schermata di Login iniziale. In questo modo il sistema è pronto se un altro utente (magari un medico) deve fare l'accesso con il proprio account.



3) Schermata Dashboard Medico

Questa schermata è la postazione di lavoro dedicata al personale medico. Viene aperta dal **Controller** subito dopo il login se l'utente è profilato come medico. L'interfaccia mostra un'agenda per consultare gli impegni e i moduli per registrare le attività cliniche sui pazienti.

A screenshot of the "Dashboard medico" web application. The interface is divided into two main sections. The left section, titled "Registrazione prestazioni", contains several input fields for recording a medical performance: "Matricola medico" (with value 200), "Codice ricovero", "Numero prestazione", "Data inizio" (26/06/2026 19:40), "Data fine" (26/06/2026 20:10), "Esito" (In attesa), "Prestazione", and "Sala operatoria". At the bottom of this section are three buttons: "Registra visita", "Registra intervento", and "Aggiorna esito". The right section, titled "Agenda medico", shows the "Data agenda" as 24/06/2026. It has three buttons: "Agenda giornaliera", "Agenda settimanale" (which is highlighted), and "Logout". Below these buttons is a list of medical appointments: "#11 - Intervento sala 1 - 24/06/2026 09:00 - esito In attesa", "#12 - Intervento sala 1 - 26/06/2026 15:00 - esito Intervento", "#13 - Visita ECG + Controllo - 26/06/2026 18:00 - esito In attesa", and "#14 - Intervento sala 2 - 27/06/2026 19:00 - esito In attesa".

3.1) Campi di Input per le Prestazioni

Nella parte della schermata dedicata all'inserimento dei dati, sono presenti diverse caselle di testo (JTextField) e menu a tendina (JComboBox) che servono a raccogliere le informazioni per registrare una visita o un intervento:

- **Matricola Medico e Codice Ricovero:** Campi in cui inserire i codici per identificare in modo univoco quale medico sta lavorando e a quale ricovero del paziente è collegata l'attività.
- **Numero Prestazione ed Esito:** Campi di testo per inserire il numero progressivo (ID) della prestazione e una casella di testo modificabile per scrivere il risultato clinico.
- **Data Inizio e Data Fine:** Campi formattati per inserire le ore e i giorni in cui si svolge l'attività.
- **Tipo Prestazione e Sala Operatoria:** Una scelta rapida per decidere se si tratta di una "Visita" o di un "Intervento". Se si seleziona "Intervento", bisogna compilare la casella per specificare il nome o il numero della **Sala Operatoria**.

3.2) Bottone "Registra Visita"

È il pulsante che invia i dati per inserire una nuova visita medica. Quando viene cliccato, il programma fa dei controlli bloccanti: verifica nel database che il medico sia regolarmente **in turno** in quella fascia oraria, che il ricovero del paziente sia ancora aperto e che il medico non abbia altre visite o interventi sovrapposti nello stesso momento. Se i controlli riescono, la visita viene salvata sul database.

3.3) Bottone "Registra Intervento"

Questo pulsante funziona esattamente come quello della visita, ma serve a salvare un intervento chirurgico. Oltre a fare i controlli standard su turni e orari, il codice Java si assicura che sia stato inserito anche il campo della sala operatoria, salvando poi tutte le informazioni nel database.

3.4) Bottone "Aggiorna Esito"

È un bottone di modifica. Permette al medico di selezionare una prestazione già fatta e di aggiornare o correggere la casella dell'**Esito** (ad esempio per inserire i risultati di un esame arrivati in un secondo momento), salvando la modifica sul database senza dover ricreare la prestazione da zero.

3.5) Tabella Agenda Giornaliera

È un pannello della GUI che mostra, all'interno di una tabella (JTable), tutte le attività che il medico deve svolgere nella giornata selezionata. Il sistema legge i dati dal database e riempie la tabella filtrando le prestazioni in base alla matricola del medico loggato e alla data odierna.



3.6) Tabella Agenda Settimanale

Simile alla precedente, questa vista tabellare permette al medico di avere un quadro visivo più ampio, mostrando tutti i turni, le visite e gli interventi pianificati per l'intera settimana. Il programma prende le prestazioni e le ordina automaticamente in ordine cronologico in base alla data e ora di inizio.

3.7) Bottone "Logout"

Posizionato in alto o in un angolo della schermata, questo pulsante chiude immediatamente la dashboard del medico, cancella la sessione corrente dalla memoria dell'applicazione e riporta l'utente alla schermata di Login iniziale (paragrafo 1), rendendo il programma pronto per un nuovo accesso.

Link repository github:

<https://github.com/OOP2026/Gruppo12/tree/main>